

```

import pandas as pd
import numpy as np

data = {
    "Employee": ["Alice", "Bob", "Charlie", "David", "Eva"],
    "Age": [25, 30, 28, np.nan, 35],
    "Department": ["HR", "IT", "IT", "Sales", "HR"],
    "Salary": [45000, 65000, 70000, 55000, np.nan]
}

df = pd.DataFrame(data)

print("Number of Rows:", df.shape[0])
print("Number of Columns:", df.shape[1])

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())

print("\nStatistical Summary:")
print(df.describe())

```

Number of Rows: 5
Number of Columns: 4

Data Types:

Employee	object
Age	float64
Department	object
Salary	float64

dtype: object

Missing Values:

Employee	0
Age	1
Department	0
Salary	1

dtype: int64

Statistical Summary:

	Age	Salary
count	4.000000	4.000000
mean	29.500000	58750.000000
std	4.203173	11086.778913
min	25.000000	45000.000000
25%	27.250000	52500.000000
50%	29.000000	60000.000000
75%	31.250000	66250.000000
max	35.000000	70000.000000

```
import pandas as pd
import matplotlib.pyplot as plt

grades = ["A", "B", "A", "C", "B", "A", "D", "B", "C", "A"]

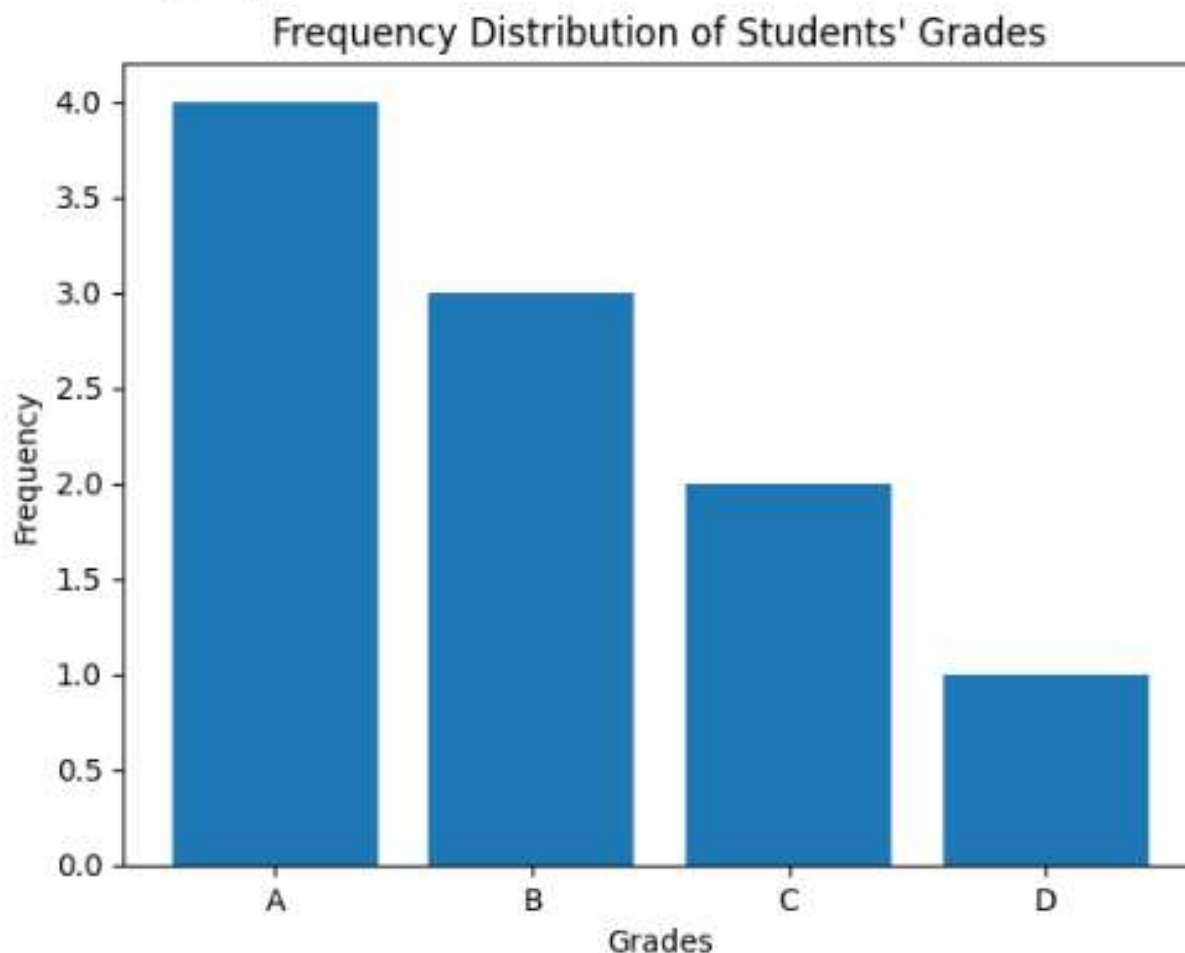
df = pd.DataFrame({"Grade": grades})

frequency = df["Grade"].value_counts().sort_index()

print("Frequency Distribution:")
print(frequency)

plt.bar(frequency.index, frequency.values)
plt.xlabel("Grades")
plt.ylabel("Frequency")
plt.title("Frequency Distribution of Students' Grades")
plt.show()
```

```
Frequency Distribution:
Grade
A      4
B      3
C      2
D      1
Name: count, dtype: int64
```



```

import pandas as pd
import matplotlib.pyplot as plt

prices = [2500000, 3200000, 4500000, 3800000, 5000000,
          4200000, 6000000, 3500000, 4800000, 7000000,
          5500000, 3000000, 3900000, 4500000, 5200000]

df = pd.DataFrame({"House_Price": prices})

print("Skewness:", df["House_Price"].skew())

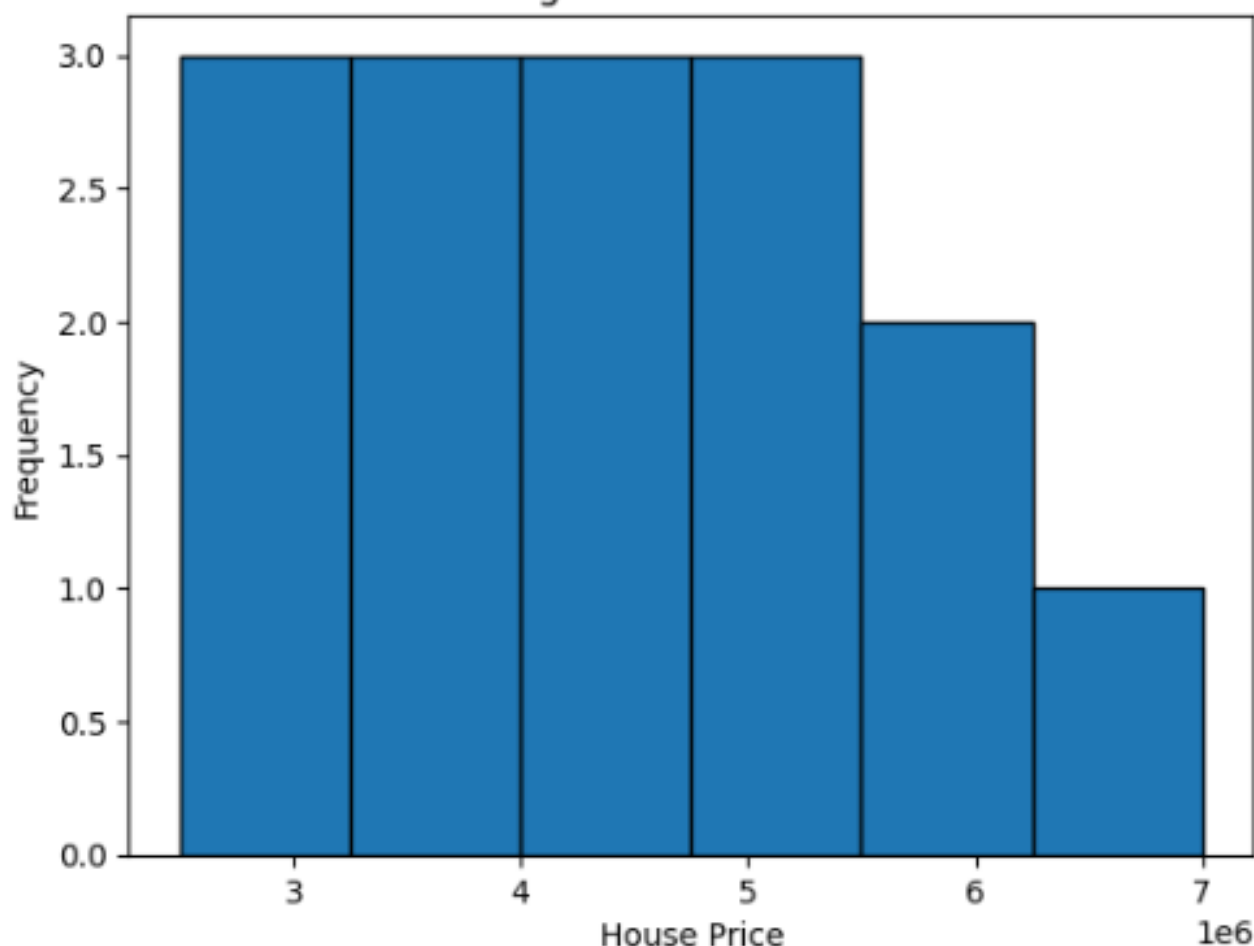
plt.hist(df["House_Price"], bins=6, edgecolor="black")
plt.xlabel("House Price")
plt.ylabel("Frequency")
plt.title("Histogram of House Prices")
plt.show()

if df["House_Price"].skew() > 0:
    print("Data is positively skewed.")
elif df["House_Price"].skew() < 0:
    print("Data is negatively skewed.")
else:
    print("Data is approximately normally distributed.")

```

Skewness: 0.4278745900583892

Histogram of House Prices



Data is positively skewed.

```

import pandas as pd
import matplotlib.pyplot as plt

salary = [30000, 35000, 40000, 45000, 50000, 55000, 60000, 65000, 70000, 150000]

df = pd.DataFrame({"Salary": salary})

plt.boxplot(df["Salary"])
plt.ylabel("Salary")
plt.title("Box Plot of Salary Data")
plt.show()

Q1 = df["Salary"].quantile(0.25)
Q3 = df["Salary"].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

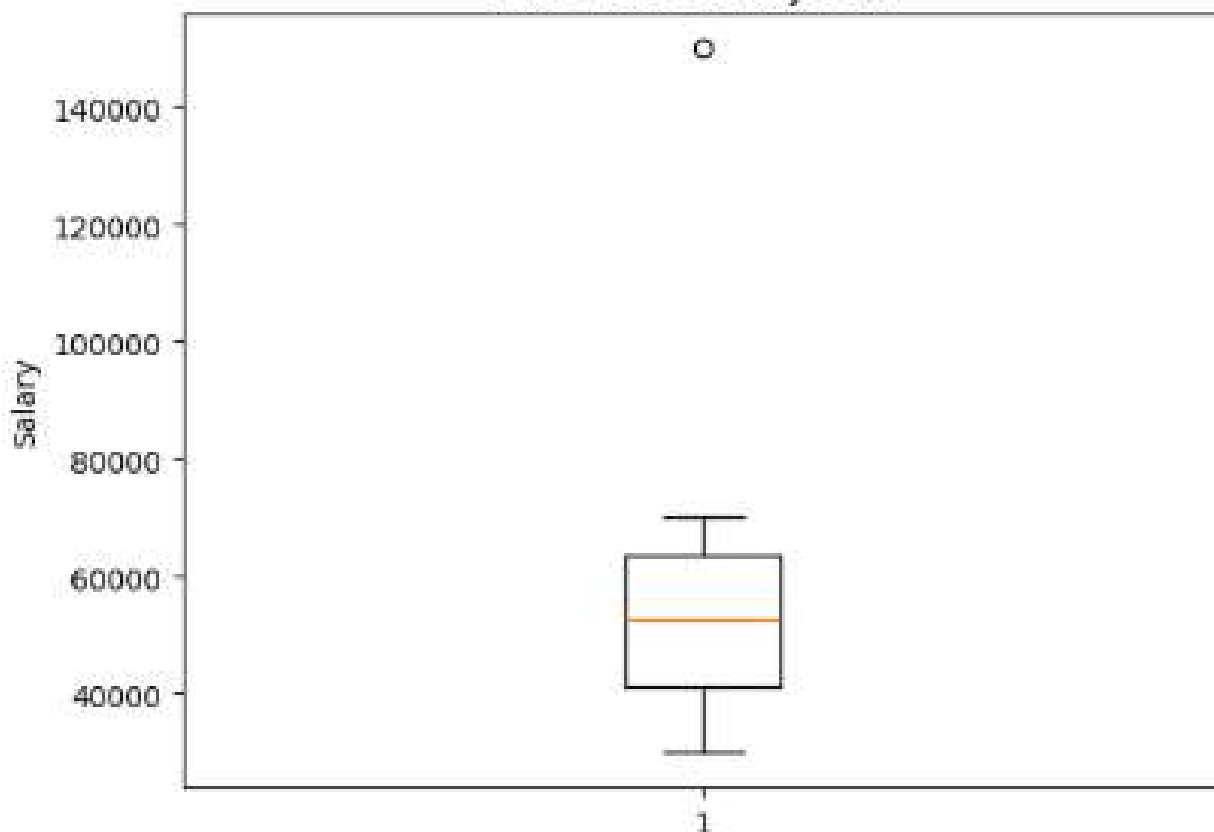
outliers = df[(df["Salary"] < lower) | (df["Salary"] > upper)]

print("Outliers:")
print(outliers)

if len(outliers) > 0:
    print("Outliers are present.")
else:
    print("No outliers are present.")

```

Box Plot of Salary Data



```

Outliers:
  Salary
9  150000
Outliers are present.

```

```
import pandas as pd

marks = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 150]

df = pd.DataFrame({"Marks": marks})

Q1 = df["Marks"].quantile(0.25)
Q3 = df["Marks"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[(df["Marks"] < lower) | (df["Marks"] > upper)]

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower)
print("Upper Bound:", upper)

print("\nOutliers:")
print(outliers)
```

```
Q1: 57.5
Q3: 71.0
IQR: 13.5
Lower Bound: 37.25
Upper Bound: 91.25
```

```
Outliers:
   Marks
10    150
```

```
import pandas as pd

sales = [1000, 1200, 1500, 1800, 2000, 2200, 2500, 2700, 3000, 15000]

df = pd.DataFrame({"Sales": sales})

Q1 = df["Sales"].quantile(0.25)
Q3 = df["Sales"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

cleaned_df = df[
    (df["Sales"] >= lower) &
    (df["Sales"] <= upper)
]

print("Original Dataset:")
print(df)

print("\nCleaned Dataset:")
print(cleaned_df)
```

Original Dataset:

	Sales
0	1000
1	1200
2	1500
3	1800
4	2000
5	2200
6	2500
7	2700
8	3000
9	15000

Cleaned Dataset:

	Sales
0	1000
1	1200
2	1500
3	1800
4	2000
5	2200
6	2500
7	2700
8	3000

```

import pandas as pd
import matplotlib.pyplot as plt

data = [100, 110, 120, 125, 130, 135, 140, 145, 150, 500]

df = pd.DataFrame({"Value": data})

Q1 = df["Value"].quantile(0.25)
Q3 = df["Value"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

cleaned_df = df[
    (df["Value"] >= lower) &
    (df["Value"] <= upper)
]

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.hist(df["Value"], edgecolor="black")
plt.title("Before Removing Outliers")
plt.xlabel("Value")
plt.ylabel("Frequency")

plt.subplot(1, 2, 2)
plt.hist(cleaned_df["Value"], edgecolor="black")
plt.title("After Removing Outliers")
plt.xlabel("Value")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.boxplot(df["Value"])
plt.title("Before Removing Outliers")

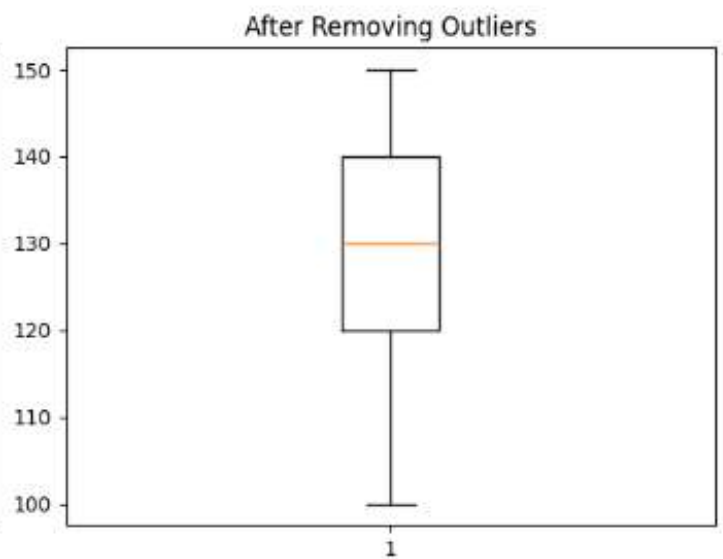
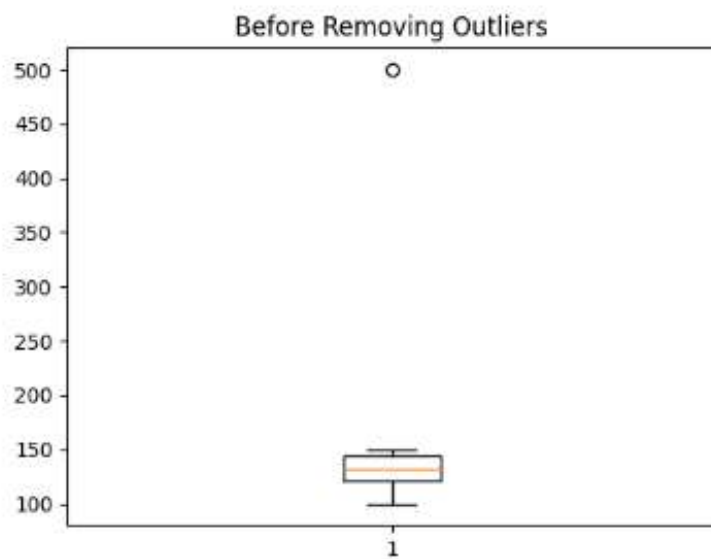
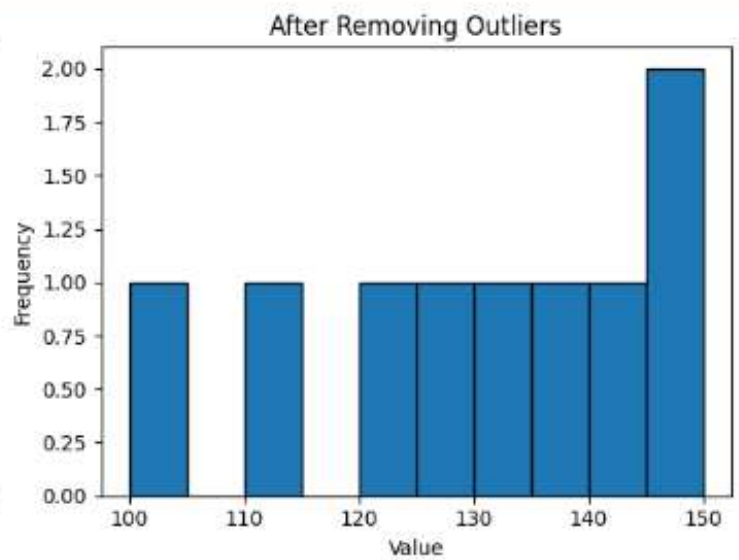
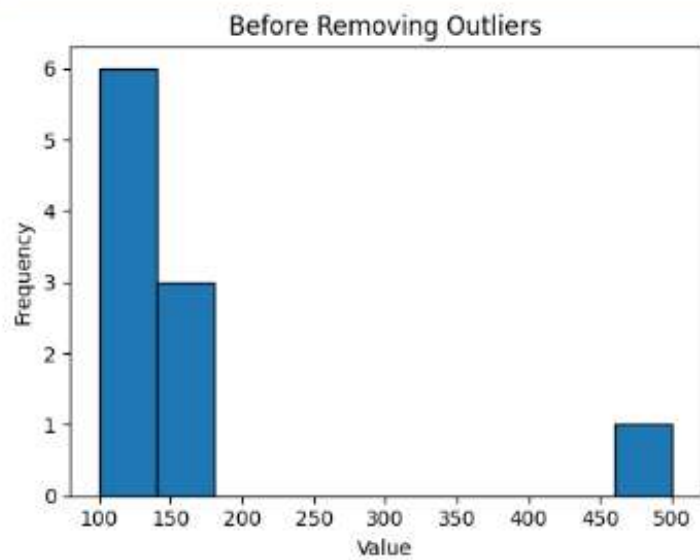
plt.subplot(1, 2, 2)
plt.boxplot(cleaned_df["Value"])
plt.title("After Removing Outliers")

plt.tight_layout()
plt.show()

print("Original Data:")
print(df)

print("\nCleaned Data:")
print(cleaned_df)

```



Original Data:

	Value
0	100
1	110
2	120
3	125
4	130
5	135
6	140
7	145
8	150
9	500

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 150 entries, 0 to 149

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	sepal length (cm)	150 non-null	float64
1	sepal width (cm)	150 non-null	float64
2	petal length (cm)	150 non-null	float64
3	petal width (cm)	150 non-null	float64
4	Species	150 non-null	int64

dtypes: float64(4), int64(1)

memory usage: 6.0 KB

None

Missing Values:

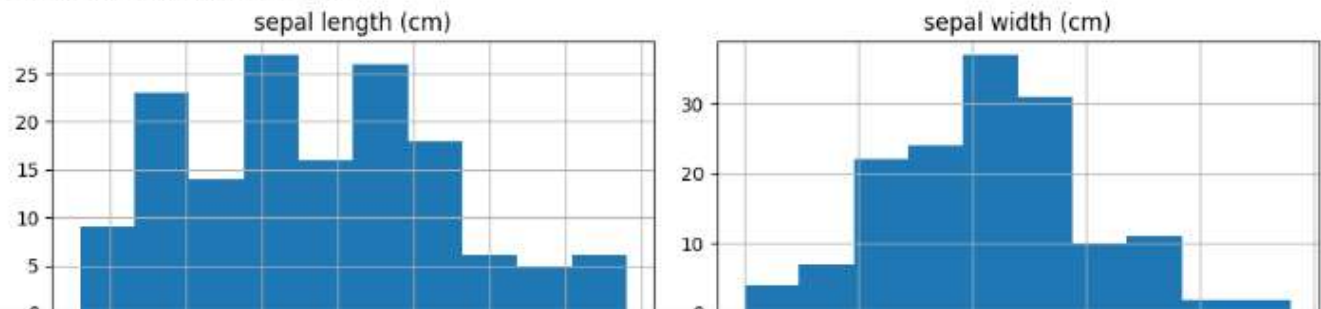
sepal length (cm) 0
sepal width (cm) 0
petal length (cm) 0
petal width (cm) 0
Species 0
dtype: int64

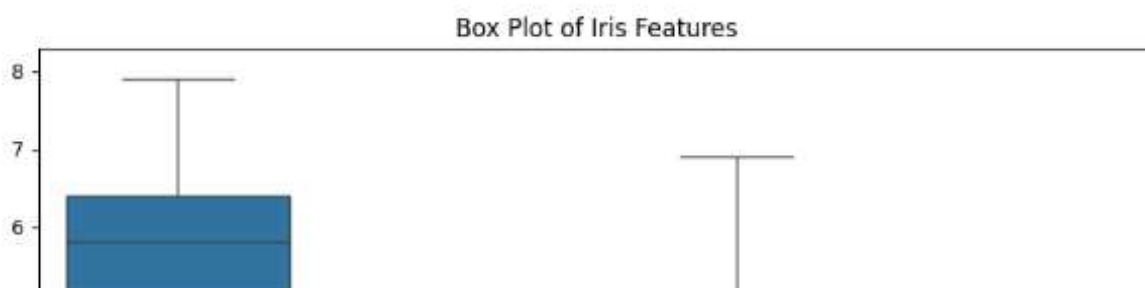
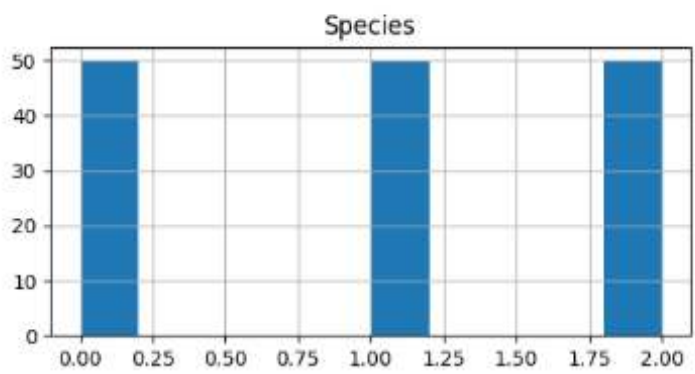
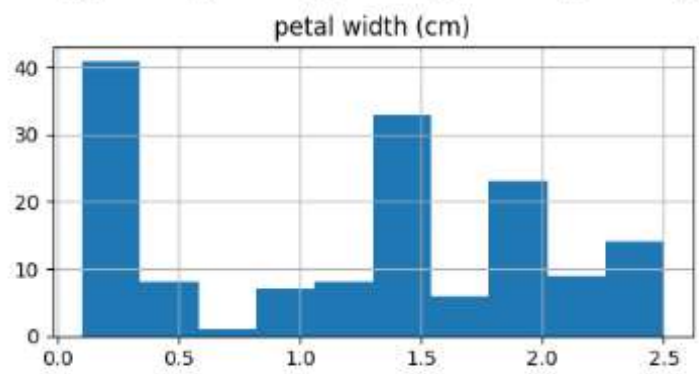
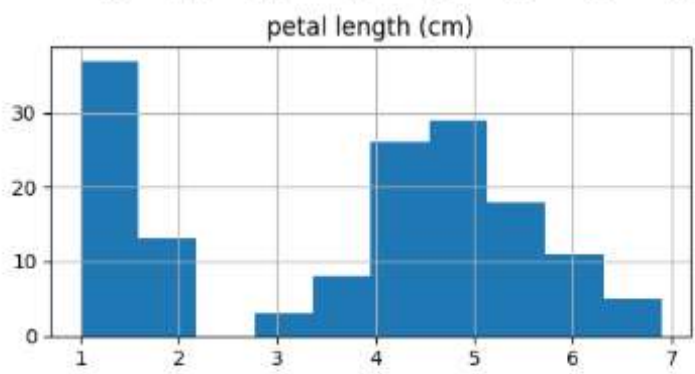
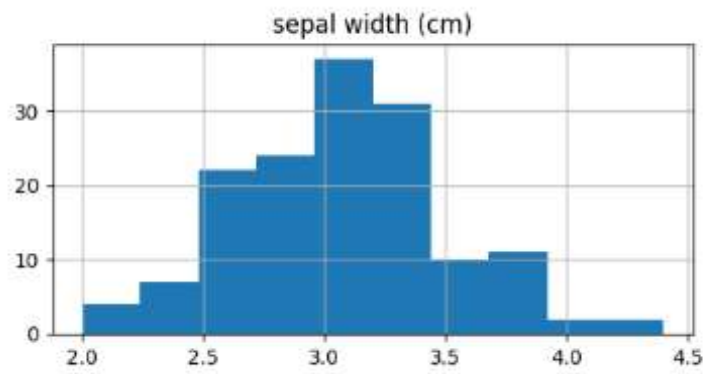
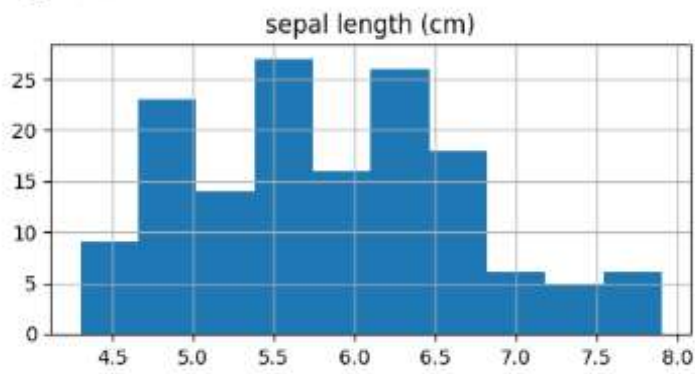
Descriptive Statistics:

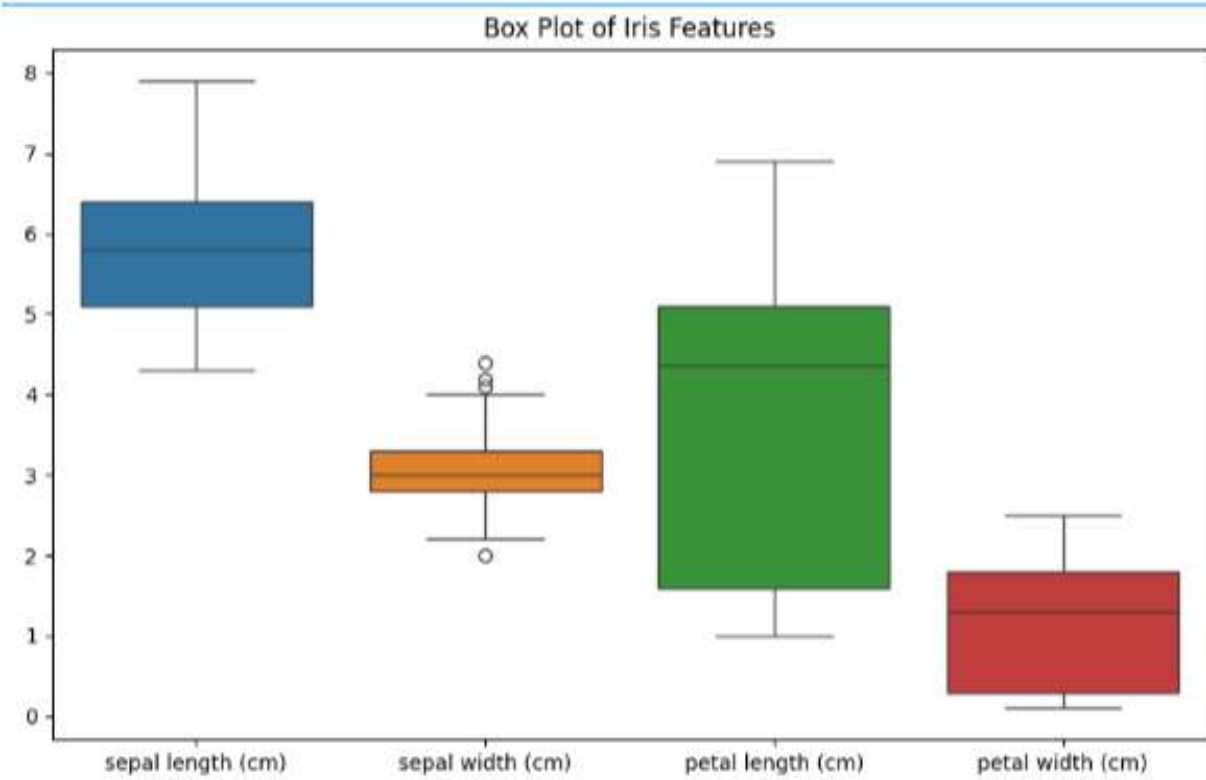
	sepal length (cm)	sepal width (cm)	petal length (cm) \
count	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000
std	0.828066	0.435866	1.765298
min	4.300000	2.000000	1.000000
25%	5.100000	2.800000	1.600000
50%	5.800000	3.000000	4.350000
75%	6.400000	3.300000	5.100000
max	7.900000	4.400000	6.900000

	petal width (cm)	Species
count	150.000000	150.000000
mean	1.199333	1.000000
std	0.762238	0.819232
min	0.100000	0.000000
25%	0.300000	0.000000
50%	1.300000	1.000000
75%	1.800000	2.000000
max	2.500000	2.000000

<Figure size 800x500 with 0 Axes>







Number of Outliers: 4

Cleaned Dataset:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	
..	...	...	...	...	
145	6.7	3.0	5.2	2.3	
146	6.3	2.5	5.0	1.9	
147	6.5	3.0	5.2	2.0	
148	6.2	3.4	5.4	2.3	
149	5.9	3.0	5.1	1.8	

	Species
0	0
1	0
2	0
3	0
4	0

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris

iris = load_iris()

df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

df["Species"] = iris.target

print("Dataset Information:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDescriptive Statistics:")
print(df.describe())

plt.figure(figsize=(8, 5))
df.hist(figsize=(10, 8))
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 6))
sns.boxplot(data=df.drop(columns=["Species"]))
plt.title("Box Plot of Iris Features")
plt.show()

Q1 = df[iris.feature_names].quantile(0.25)
Q3 = df[iris.feature_names].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = ((df[iris.feature_names] < lower) |
            (df[iris.feature_names] > upper)).any(axis=1)

print("\nNumber of Outliers:", outliers.sum())

cleaned_df = df[~outliers]

print("\nCleaned Dataset:")
print(cleaned_df)

cleaned_df.to_csv(
    r"C:\Users\admin\Downloads\FOD jothika\iris_cleaned.csv",
    index=False
)

print("\nCleaned dataset saved successfully.")

```